

SISTEMA DE GESTÃO DE BANCO DE ALIMENTOS

Documento de Arquitetura e Requisitos

Atividade de Estudo Programada (AEP) — 1ª Entrega (1º Bimestre)

Objetivos de Desenvolvimento Sustentável

ODS 2 — Fome Zero e Agricultura Sustentável • ODS 12 — Consumo e Produção Responsáveis

Equipe	RA
Bruno André Fernandes Ortega	25291321-2
Giovanna Cristina Martins	25001909-2

1. Descoberta e Concepção — O Problema e a ODS

Bancos de alimentos e organizações do terceiro setor (ONGs) que recebem doações de mercados, feiras e indústrias enfrentam uma dificuldade operacional real: controlar o que entra no estoque, o prazo de validade de cada item recebido e para quem cada doação foi efetivamente distribuída. Na maioria dessas organizações o controle é feito por planilhas manuais e desconexas, o que se torna uma das principais causas de **desperdício de alimentos perecíveis** — itens que vencem antes de chegar a quem precisa — além de comprometer a **rastreabilidade** das entregas já realizadas.

Partes Interessadas e Dores Identificadas

- **Operador do banco de alimentos** (usuário principal): precisa saber, a qualquer momento, o que há em estoque e o que está prestes a vencer, para decidir a ordem de saída.
- **Doadores** (pessoa física, mercado ou indústria): querem que a doação seja aproveitada, e não perdida por falta de controle.
- **Beneficiários**: dependem do alimento chegar em condições adequadas de consumo.

Resumo do Discovery (Matriz CSD e Jornada do Usuário)

Durante a fase de descoberta, ao mapear a rotina do **Operador**, identificou-se que o maior ponto de perda não está na captação de doações, mas na **falta de critério de saída**: sem um sistema que priorize a distribuição, torna-se difícil identificar rapidamente os alimentos próximos do vencimento, o que aumenta o risco de desperdício e compromete o histórico das doações realizadas. A Matriz CSD (Certezas, Suposições e Dúvidas) organizou essas necessidades e direcionou o escopo do sistema para três eixos: **controle de estoque**, **priorização de itens conforme a proximidade do vencimento** e **registro rastreável das distribuições**.

Alinhamento com os ODS da ONU

O projeto alinha-se diretamente ao **ODS 2 (Fome Zero e Agricultura Sustentável)**, em especial à **meta 2.1** — garantir o acesso de todas as pessoas a uma alimentação segura — e ao **ODS 12 (Consumo e Produção Responsáveis)**, em especial à **meta 12.3**, que trata da redução do desperdício de alimentos. Ao automatizar o controle de estoque com priorização por validade, o sistema reduz diretamente o desperdício e aumenta a quantidade de alimento que efetivamente chega aos beneficiários, transformando um processo hoje manual e sujeito a perdas em um fluxo controlado e rastreável.

2. Lista de Requisitos Funcionais (Escopo)

A entidade central do sistema é o **Alimento**, sobre a qual se aplicam as operações completas de CRUD. Os requisitos abaixo descrevem, de forma direta, o que o sistema deve fazer para atender às dores levantadas na descoberta.

ID	Requisito Funcional
RF01	O sistema deve permitir o cadastro de doadores, contendo nome, tipo (pessoa física, mercado ou indústria) e contato.
RF02	O sistema deve permitir registrar a entrada de um alimento no estoque — perecível ou não perecível — vinculado a um doador, contendo nome, quantidade, data de recebimento e, quando perecível, data de validade.
RF03	O sistema deve listar os alimentos em estoque ordenados por prioridade de distribuição, calculada de forma distinta conforme o tipo de alimento (o item mais próximo do vencimento é distribuído primeiro).
RF04	O sistema deve permitir o cadastro de beneficiários, contendo nome e contato.
RF05	O sistema deve permitir registrar a distribuição de uma quantidade de um alimento a um beneficiário, dando baixa automática na quantidade em estoque.
RF06	O sistema deve alertar, ao iniciar, quais alimentos perecíveis estão a poucos dias do vencimento.
RF07	O sistema deve gerar um relatório de desperdício, listando os alimentos vencidos que não foram distribuídos, consultável pelo console.

Observação de escopo: RF01–RF05 constituem o núcleo obrigatório (cadastros, priorização polimórfica e distribuição com baixa de estoque). RF06 e RF07 são requisitos de valor agregado, priorizados na fase final do 2º bimestre.

3. Justificativa Técnica e Arquitetural

Linguagem de Programação — Java (JDK 17 LTS)

O sistema será desenvolvido em **Java**. A escolha fundamenta-se na sua **tipagem estática e forte**, que reduz erros em tempo de execução, e na sua implementação madura e explícita dos **pilares da Orientação a Objetos**. O domínio do problema exige regras de negócio distintas por tipo de item — o cálculo de prioridade de um alimento perecível difere estruturalmente do de um não perecível — o que torna **herança** e **polimorfismo** soluções naturais, e não artificiais, para o modelo. Abstração e encapsulamento garantem que essas regras fiquem coesas e testáveis, em aderência direta às exigências da disciplina.

Persistência — MySQL via JDBC

Para a persistência será adotado o banco de dados relacional **MySQL**, acessado por **JDBC** através do conector oficial **MySQL Connector/J** (`com.mysql.cj.jdbc.Driver`), sem uso de ORM. A escolha do MySQL alinha-se diretamente ao conteúdo praticado na disciplina de **Banco de Dados** do semestre, o que reduz a curva de aprendizado da equipe e permite reaproveitar convenções já dominadas (DDL, chaves e JOINS). O modelo relacional assegura a **integridade referencial** entre as entidades Doador, Alimento, Beneficiário e Distribuição por meio de chaves estrangeiras, e suporta as **operações agregadas** (somatórios de quantidade distribuída, filtragem por data de validade) necessárias aos relatórios. A opção por JDBC puro é deliberada: mantém o controle explícito do SQL, evidencia o domínio da disciplina de Banco de Dados e mantém o projeto dentro do escopo técnico do semestre.

Padrão Arquitetural — Camadas (MVC adaptado ao console)

A aplicação adota uma arquitetura **em camadas**, com responsabilidades separadas:

Camada	Responsabilidade
Model (Domínio)	Classes de negócio e a hierarquia polimórfica de Alimento. Concentra as regras de negócio.
DAO / Repository	Camada de acesso a dados: encapsula todo o SQL e a comunicação JDBC com o MySQL.
Service	Orquestra as regras de negócio (ex.: baixa de estoque ao registrar distribuição).
View (Console)	Menu interativo via terminal — entrada e saída padrão, sem interface gráfica.

A separação em camadas mantém o SQL isolado na camada DAO e as regras de negócio na camada de domínio/serviço, sustentando o escopo funcional (RF01–RF07) com baixo acoplamento e alta coesão. Como a aplicação é orientada a console, não há necessidade de camada de API ou front-end, o que concentra o esforço da equipe na qualidade do modelo de dados e das regras de negócio — exatamente o foco avaliado no curso de Engenharia de Software.

Ambiente de execução: instruções completas de compilação e execução (versão do JDK, criação do banco, execução do script SQL e configuração de conexão) estarão no arquivo README.md do repositório, respeitando a compatibilidade de ambientes discutida na disciplina de Sistemas Operacionais.

4. Estruturação do GitHub e Diagramas

4.1 Repositório de Controle de Versão

Repositório público no GitHub, contendo a estrutura de diretórios /src, /docs e /database, com o arquivo README.md espelhando a lista de requisitos e o cronograma deste documento.

Link do repositório: <https://github.com/BrunoFernandes0/banco-de-alimentos-aep>

4.2 Diagrama de Classes (UML)

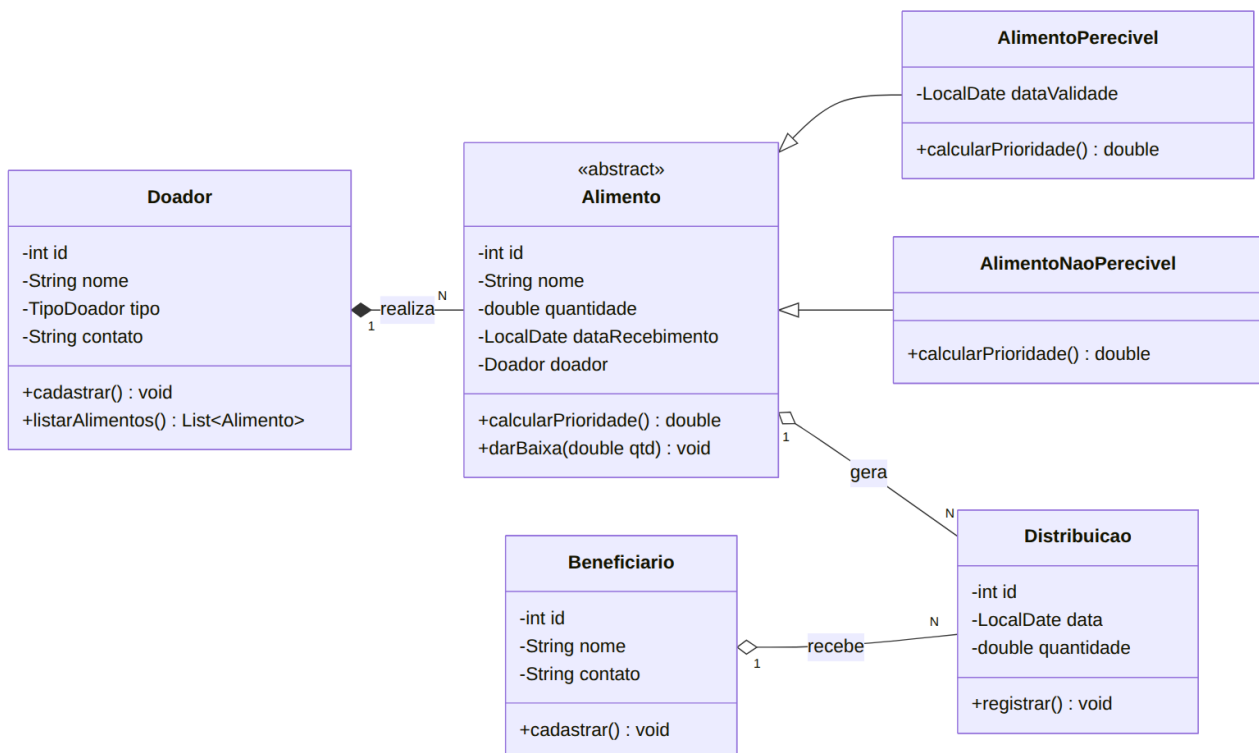


Figura 1 — Diagrama de classes do sistema.

O diagrama atende explicitamente aos três elementos exigidos:

- **Herança:** a superclasse abstrata **Alimento** (atributos comuns: id, nome, quantidade, dataRecebimento e o doador de origem) é especializada por **AlimentoPecivel** e **AlimentoNaoPecivel**, reaproveitando estado e comportamento comuns.
- **Composição 1:N:** a classe **Doador** compõe uma coleção de **Alimento** (um doador realiza várias doações — relação 1:N, marcada com losango de composição). Complementarmente, **Alimento** e **Beneficiario** agregam múltiplas instâncias de **Distribuicao**.
- **Polimorfismo:** o método **calcularPrioridade()** é declarado abstrato em **Alimento** e sobrescrito (`@Override`) de formas distintas: em **AlimentoPecivel** a prioridade cresce conforme a proximidade da data de validade; em **AlimentoNaoPecivel** permanece fixa e mais baixa. Isso permite ordenar o estoque de forma polimórfica, invocando o mesmo método independentemente do tipo concreto.

4.3 Diagrama do Banco de Dados (DER)

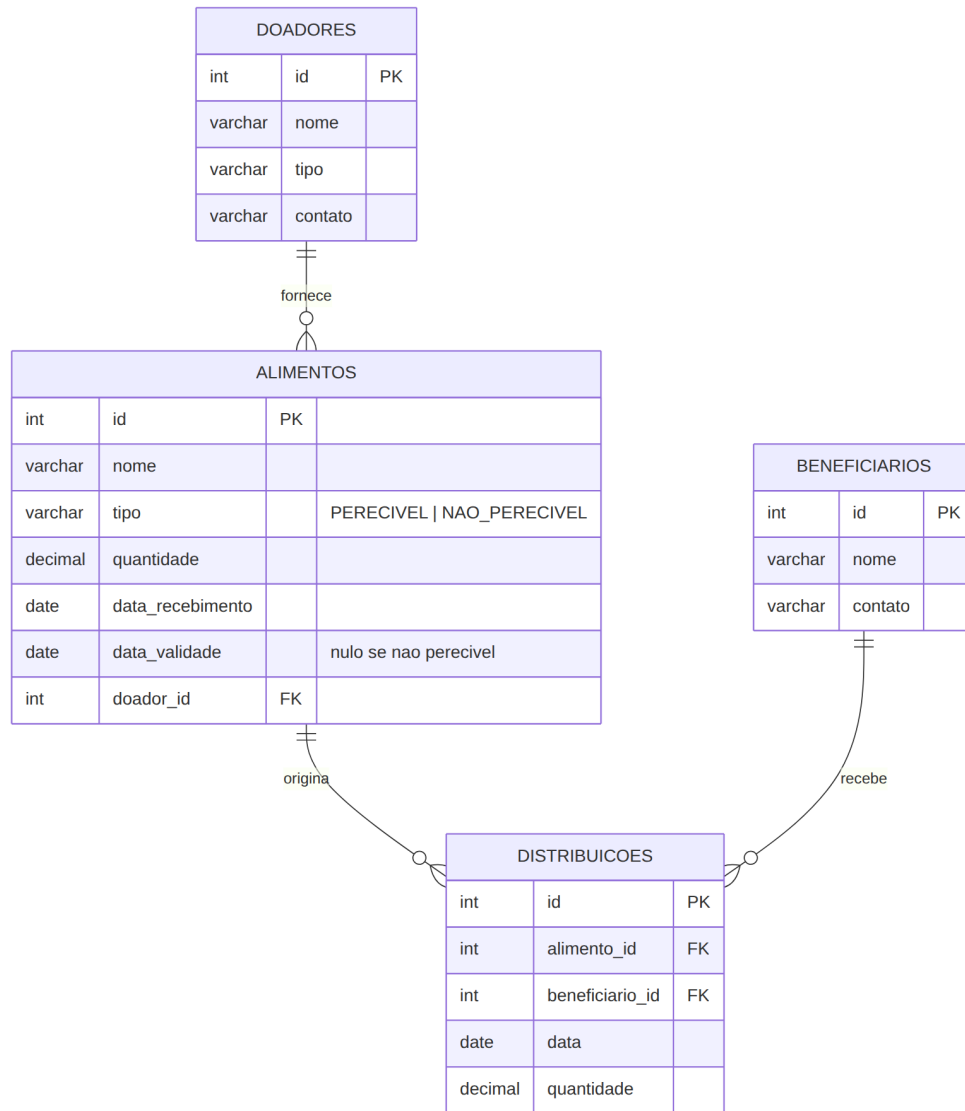


Figura 2 — Diagrama Entidade-Relacionamento (DER).

- **doadores** — dados de quem realiza as doações (pessoa física, mercado ou indústria).
- **alimentos** — itens recebidos, com coluna discriminadora **tipo** (PERECIVEL / NAO_PERECIVEL) e chave estrangeira **doador_id**. O campo **data_validade** é nulo para itens não perecíveis.
- **beneficiarios** — pessoas ou famílias atendidas pelo banco de alimentos.
- **distribuicoes** — cada entrega realizada, com as chaves estrangeiras **alimento_id** e **beneficiario_id**, **data** e **quantidade**, permitindo a baixa automática no estoque e o histórico completo de distribuição.

5. Planejamento Ágil — Cronograma / Backlog (2º Bimestre)

Fatiamento vertical das entregas do 2º bimestre. O histórico de commits no GitHub refletirá a divisão de responsabilidades entre os dois integrantes.

Sprint / Data	Épico	User Story	Resp.
Sprint 1 15/09–28/09	Modelagem e Base	COMO equipe, QUERO estruturar o DER, o script SQL de criação das tabelas e a conexão JDBC, PARA QUE haja uma base sólida para as demais funcionalidades.	Bruno
Sprint 2 29/09–12/10	Cadastros Base	COMO operador, QUERO cadastrar doadores e registrar a entrada de alimentos (perecíveis ou não), PARA QUE o estoque comece a ser controlado.	Giovanna
Sprint 3 13/10–26/10	Núcleo Polimórfico	COMO operador, QUERO visualizar o estoque ordenado por prioridade de distribuição e ter o CRUD completo de Alimento, PARA QUE os itens mais próximos do vencimento saiam primeiro.	Bruno
Sprint 4 27/10–09/11	Beneficiários e Distribuição	COMO operador, QUERO cadastrar beneficiários e registrar distribuições com baixa automática de estoque, PARA QUE o histórico de entregas fique rastreável.	Giovanna
Sprint 5 10/11–14/11	Alertas, Relatórios e Entrega	COMO usuário, QUERO alertas de vencimento, relatório de desperdício, o menu integrado e o README de execução, PARA QUE a aplicação rode de ponta a ponta.	Ambos

Nota: a data de entrega final do 2º bimestre é provisória (prevista para meados de novembro) e será ajustada conforme o calendário oficial. Alterações de escopo, se necessárias, serão registradas no README, preservando a coerência da solução.